

Aksaray Üniversitesi
Yazılım Mühendisliği Bölümü
Yazılım Mühendisliğinde Uygulamalar I Dersi
Proje Raporu

Ad: Süleyman

Soyad: Meral

Numara:220211033

Öğretim Görevlisi: Doç.Dr. Bilal Şenol

Proje Adı: Yurt Yönetim Sistemi



Proje Özeti

Bu projenin amacı, yurt yönetimi süreçlerini dijitalleştirerek yöneticilerin, öğrencilerin ve personelin iş yükünü azaltmak, verimliliği artırmak ve veri bütünlüğünü sağlamaktır. Geleneksel yöntemlerde öğrenci oda yerleşimleri, yoklamalar, aidat takibi ve bakım talepleri gibi işlemler manuel yapıldığı için hataya açık, zaman alıcı ve denetlenmesi zor olmaktadır. Bu sorunlara çözüm olarak proje kapsamında, katmanlı mimari (Clean Architecture) yaklaşımıyla geliştirilen bir web tabanlı Yurt Yönetim Sistemi tasarlanmıştır. Uygulama Entity Framework Core ve ASP.NET Core Web API, React teknolojileri kullanılarak inşa edilmiştir. Sistem; yöneticilerin oda, öğrenci, kullanıcı ve bakım işlemlerini tek bir arayüzden yönetebilmesini; öğrencilerin ise kendi oda bilgilerini ve taleplerini görüntüleyebilmesini sağlar. Elde edilen sonuçlar, yönetim süreçlerinin dijital ortamda merkezi hale getirilmesiyle birlikte; veri erişim hızının arttığını, hata oranının azaldığını ve kullanıcı deneyiminin önemli ölçüde iyileştiğini göstermektedir.

İçindekiler Tablosu

Projenin Özeti	2
İçindekiler	3
Projenin Amacı ve Kapsamı.....	4
Hedef Kitle	5
Gereksinimler	6
Sistem Mimarisi ve Tasarımı(Teknik Detaylar)..	11
Sistemde Kullanılması Planlanan Teknolojiler ..	17
Sistemde Kullanılması Planlanan Tasarım Desenleri.....	18
Veritabanı Şeması.....	20
Örnek Kod Yapıları.....	21

Projenin Amacı ve Kapsamı

Bu projenin amacı, yurt yönetiminde karşılaşılan manuel ve dağınık süreçleri dijitalleştirerek daha verimli, hatasız ve izlenebilir bir sistem oluşturmaktır. Geleneksel yöntemlerde öğrenci kayıtları, oda yerleşimleri, yoklamalar, aidat takibi ve bakım talepleri gibi işlemler farklı dosyalar veya formlar üzerinden manuel olarak yürütülmektedir. Bu durum; veri kayıplarına, bilgiye geç erişime ve yönetsel karmaşaya yol açmaktadır. Bu ihtiyaçtan yola çıkılarak geliştirilen sistem, yurt yöneticilerinin ve öğrencilerin işlemlerini tek bir platform üzerinden yönetebilmesini sağlamayı hedeflemektedir.

Hedef Kitle

Geliştirilen Yurt Yönetim Sistemi, öğrenci yurtlarında yürütülen operasyonel süreçleri dijitalleştirmeyi, hızlandırmayı ve hataya açık manuel işlemleri en aza indirmeyi amaçlamaktadır. Bu doğrultuda sistemin birincil hedef kitlesi; yurt yönetimi, idari personel, danışma görevlileri ve öğrenci işleri birimleridir. Bunun yanında sistem, yurtlarda konaklayan öğrencilerin bilgi erişimini kolaylaştırarak daha şeffaf ve düzenli bir iletişim ortamı sunmayı hedeflemektedir. Dolayısıyla öğrenciler de dolaylı hedef kitleyi oluşturmaktadır. Sonuç olarak Yurt Yönetim Sistemi; yurt yöneticilerinden idari personele, güvenlik ve danışma görevlilerinden öğrencilere kadar yurt süreçlerinin tüm paydaşlarına hitap eden, kapsamlı ve sürdürülebilir bir dijital çözüm sağlamayı amaçlamaktadır.

Gereksinimler

➤ Fonksiyonel Gereksinimler:

▪ Kullanıcı İşlemleri:

- Yönetici yeni öğrenci, personel ve oda bilgilerini sisteme ekleyebilmelidir.
- Yönetici öğrencilerin ve personellerin bilgilerini güncelleyebilmelidir.
- Yönetici öğrencilerin ve personellerin kaydını silbilmelidir.
- Kullanıcılar sisteme kullanıcı adları ile giriş yapabilmelidir.
- Şifre unutulması durumunda , kullanıcılar kullanıcı adları ile şifrelerini mail yolu ile sıfırlayabilmelidir.

▪ Oda İşlemleri:

- Yönetici oda oluşturabilmeli,güncelleyebilmeli ve silbilmelidir.
- Yönetici odalara öğrenci atayabilmelidir.
- Yönetici oda listesini ve oda detaylarını görebilmelidir.

▪ Öğrenci İşlemleri:

- Öğrencilerin okul, iletişim ve kişisel bilgileri sistemde görüntülenebilmelidir.

■ Arıza İşlemleri:

- Öğrenciler detayı ile birlikte arıza kaydı oluşturabilmelidir.
- Yönetici, arızaları detayı ve tarihi ile birlikte görüntüleyebilmelidir.
- Yönetici arıza kayıtlarını iptal edebilmeli veya onay verebilmelidir.
- Öğrencilerin okul, iletişim ve kişisel bilgileri sistemde görüntülenebilmelidir.

■ Duyuru İşlemleri:

- Yönetici sistemden duyuru oluşturabilmelidir.
- Oluşturulan duyurular, duyurular sayfasında listelenebilmelidir.
- Duyuru oluşturulduğu anda, duyurunun tarihi başlığı ve açıklaması yurda ait telegram veya whatsapp grubuna harici olarak bir işlem yapılmadan iletilmelidir.
- Yönetici duyuruları silebilmeli ve güncelleyebilmelidir.

■ Öğrenci İzni İşlemleri:

- Öğrenciler izin, tarih aralığı ve izin açıklamalarını belirterek izin talebi oluşturabilmelidir.
- Oluşturulan izinleri yönetici görebilmelidir.

- Yöneticiler talep açılan izinleri iptal edebilmeli veya onaylayabilmelidir.
- İzin onaylandığı anda öğrencinin velisine mail veya sms gönderilmelidir.
- Öğrenci izinden döndüğü zaman sistemden izni sonlandırabilmelidir.
- Bitiş tarihi yaklaşıp, sonlandırılmayan izinler yöneticiye bildirilebilmelidir
- İzin bitiş tarihi geçmiş ve sonlandırılmamış izinler öğrencinin velisine bildirilebilmelidir

■ **Etkinlik İşlemleri:**

- Yönetici sisteme tarih,saat ve detay gibi bilgiler ile birlikte etkinlik ekleyebilmelidir.
- Eklenen bu etkinlikler öğrencilere sms veya mail yolu ile iletilebilmelidir.
- Etkinlikler, etkinlikler sayfasında listelenebilmelidir.
- Yönetici etkinleri iptal edebilmeli veya güncelleyebilmelidir.
- İptal edilen etkinliklerin bildirimi öğrencilere iletilebilmelidir.

■ **Yurt Bilgileri İşlemleri:**

- Yönetici yurda ait temel bilgileri(adres,telefon,mail,isim vb.) güncelleyebilmelidir.

▪ **Yemek İşlemleri:**

- Yönetici yemek menüsünü tarihe göre oluşturabilmelidir.
- Oluşturulan yemek menüleri öğrenciler tarafından görüntülenebilmelidir.
- Yönetici yemek menülerini güncelleyebilmeli ve silebilmelidir.

➤ **Fonksiyonel Olmayan Gereksinimler**

▪ **Performans:**

- Verilerin hızlı çekilebilmesi için caching mekanizması kullanılmalıdır.

▪ **Güvenlik:**

- Kullanıcıların parolaları şifrelenerek saklanmalıdır.
- Her kullanıcı her yapıya erişebilmemelidir.
- Sisteme kayıt olmayan kişiler etkinlik , yemek menüsü gibi detayları görüntüleyebilmemelidir.

▪ **Kullanılabilirlik:**

- Arayüz herkesin kullanabileceği kullanıcı dostu bir arayüz olmalıdır.
- Mobil ve masaüstü cihazlarda düzgün çalışmalıdır.

▪ **Bakım kolaylığı:**

- Kod yapısı modüler olmalıdır. Yeni özellikler geliştiricileri yormadan kolaylıkla eklenebilmelidir.

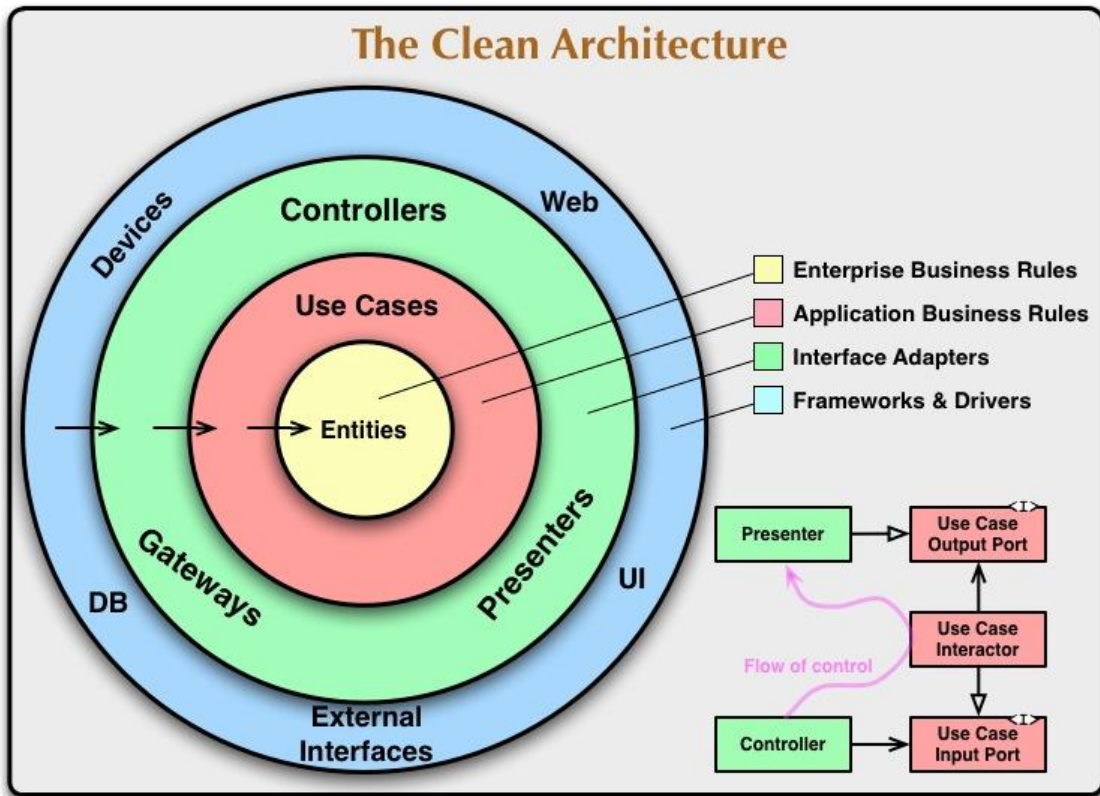
▪ **Veri Bütünlüğü:**

- Sisteme kayıtlı kullanıcılar tekrar kayıt olmamalıdır.

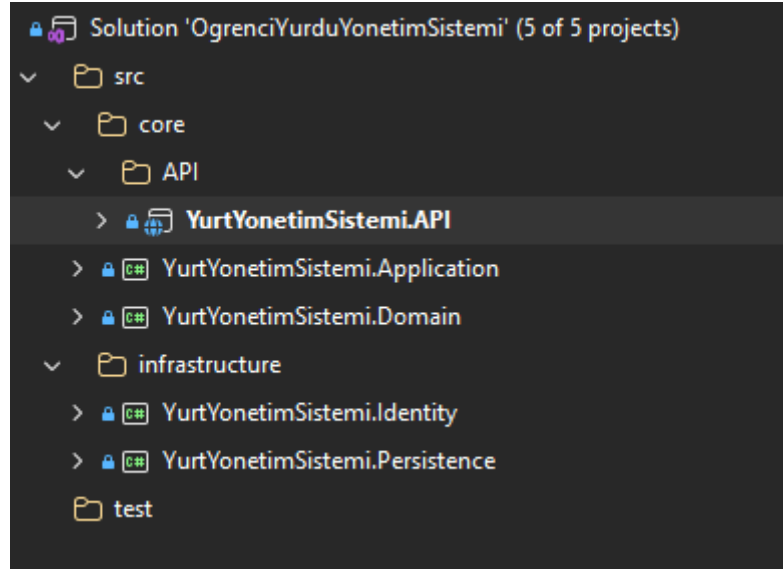
Sistem Mimarisi ve Tasarımı (Teknik Detaylar)

Mimari Yaklaşım:

Projenin backend kısmı geliştirilirken Clean Architecture yaklaşımı tercih edilmiştir. Bu mimari model, bağımlılıkların dış katmanlardan iç katmanlara doğru yöneldiği, yüksek bağımsızlık, test edilebilirlik ve esneklik sağlayan bir yapıdır. Bu sayede sistemin farklı katmanları birbirinden bağımsız olarak geliştirilebilir, test edilebilir ve gerektiğinde farklı teknolojilerle kolayca değiştirilebilir.



Mimari Yapı:



- **Core**

- ◆ **API**

- * Uygulamanın endpointler ile dış dünyaya açıldığı katmandır.
- * Controller yapıları burada bulunur.
- * Örnek endpointler aşağıdaki görselde bulunmaktadır.



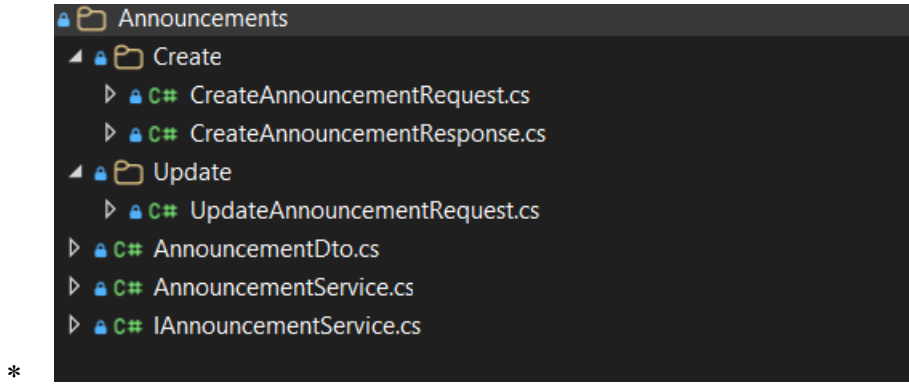
◆ **YurtYonetimSistemi.Domain:**

- * Uygulamanın entity(varlık) sınıfları, exceptionlar burada yer almaktadır.
- * Bu katman hiçbir dış bağımlılığa sahip değildir ve sistemin “kalbini” oluşturur.
- * Projenin başlangıç aşamasındaki varlıkların listesi aşağıda yer almaktadır. Proje büyüdükçe bu varlıkların sayısı artacaktır.

- (1) **Activity**
- (2) **Announcement**
- (3) **Dorm**
- (4) **Fault**
- (5) **Meal**
- (6) **Menu**
- (7) **Room**
- (8) **Staff**
- (9) **Student**
- (10) **Student Leave**

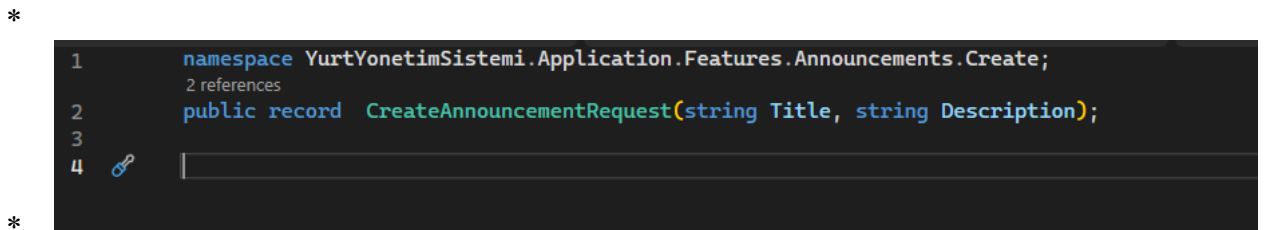
◆ **YurtYonetimSistemi.Application:**

- * İş kuralları use case’ler, veri transfer nesneleri (DTO) ve servis arabirimleri burada tanımlanır.
- * Uygulamanın business işlerinden sorumlu katmandır.
- * Repository arayüzleri de bu katmanda bulunur.(Bu opsiyonel bir tercihtir, domain katmanında da bulunabilir.)



Görsel_1

- * Yukarıdaki bulunan Görsel 1 'de örnek olarak Announcement feature yapısı verilmiştir.
- * Create ve Update Use case'i klasörlendirilip, ilgili use case'e ait olan DTO yapıları bu klasörün içerisinde yer almaktadır.



Görsel_2

- * DTO yapıları Görsel_2 de belirtildiği gibidir. DTO yapıları class type yerine record type olarak tercih edilmiştir. Bunun sebebi record type'ların immutable(değiştirilemez)olmasıdır. Requestten alınan bilgiler sadece 1 kere set edilip veri güvenliği korunacaktır.

- **Infrastructure:**

- ◆ **YurtYonetimSistemi.Persistence:**

- * Veri tabanı erişimi (EF Core) burada gerçekleştirilir.
 - * Application katmanında bulunan repository arayüzlerinin somut implemantasyonu burada gerçekleştirilir.
 - * Amaç, veri erişimini soyutlayarak veritabanına doğrudan bağımlılığı ortadan kaldırmak ve uygulamayı daha esnek ve test edilebilir hale getirmektir.
 - * Veri tabanı işlemlerinden burası sorumludur.
 - * Entity Framework Core (EF Core) kullanılarak SQL Server veritabanı ile iletişim kurar.
 - * CRUD (Create, Read, Update, Delete) işlemleri bu katmanda gerçekleştirilir.
 - * Application katmanında tanımlı repository arayüzleri burada somut sınıflarla (örneğin StudentRepository, RoomRepository) uygulanır.
 - * Her repository sınıfı DbContext üzerinden veritabanı işlemlerini yürütür.

- ◆ **YurtYonetimSistemi.Identity:**

- * Uygulamanın kullanıcı işlemlerinden sorumludur.
- * EF Core identity paketini kullanır.
- * Login, Register gibi işlemlerin arayüzleri Application katmanında oluşturulur ve somut olarak UserService üzerinden implemente edilir.
- * UserService Identity paketi ile gelen hazır metotların bulunduğu özel bir sınıftır.
- * Rol yetkilendirmeleri de bu katmanda yapılır.

◆ **YurtYonetimSistemi.Caching:**

- * Caching Katmanı, sistemde sık kullanılan verilerin geçici olarak bellekte saklanmasını sağlayan katmandır. Bu sayede veritabanına yapılan sorgu sayısı azaltılır, sistemin performansı ve yanıt süresi önemli ölçüde iyileştirilir.
- * Projenin başlangıç seviyesinde bu işlem için InMemoryCaching kullanılacaktır.
- * Proje büyürse redis altyapısına geçilebilir.

Sistemde Kullanılması Planlanan Teknolojiler:

◆ Backend:

- * ASP.Net Core Web API (.Net 8.0,C#)
- * Entity Framework Core(ORM)
- * Paketler:
 - (1) AutoMapper(Otomatik Entity-Dto Dönüşümü)
 - (2) Microsoft.AspNetCore.Identity.EntityFrameworkCore(Kullanıcı İşlemleri)
 - (3) FluentValidation.AspNetCore (Validasyon işlemleri)
- * POSTMAN(API Test Aracı)

◆ Frontend:

- * React
- * Bootstrap

Sistemde Kullanılması Planlanan Tasarım Desenleri(Design Patterns):

▪ Repository Design Pattern:

- Veri erişim işlemlerini (CRUD) iş katmanından soyutlayarak kodun daha düzenli, tekrar kullanılabilir ve test edilebilir olmasını sağlar

▪ Unit of Work Pattern:

- Birden fazla veri işlemini tek bir işlem (transaction) altında toplayarak hepsinin birlikte başarılı olmasını veya tamamen geri alınmasını(rollback) sağlar

▪ Result Pattern:

- Bir metodun sadece veri değil aynı zamanda başarı / başarısızlık durumunu, hata mesajlarını ve ek bilgileri tek bir yapı içinde döndürmesini sağlar. Örnek yapı “Görsel_3” adlı görselde verilmiştir.
- Result Pattern yapısı projenin “Application” katmanındaki “ServiceResult” classında bulunmaktadır.

▪ Options Pattern:

- Uygulama yapılandırma ayarlarını (örn. appsettings.json) tip güvenli sınıflara dönüştürerek kolayca yönetmeyi sağlar.

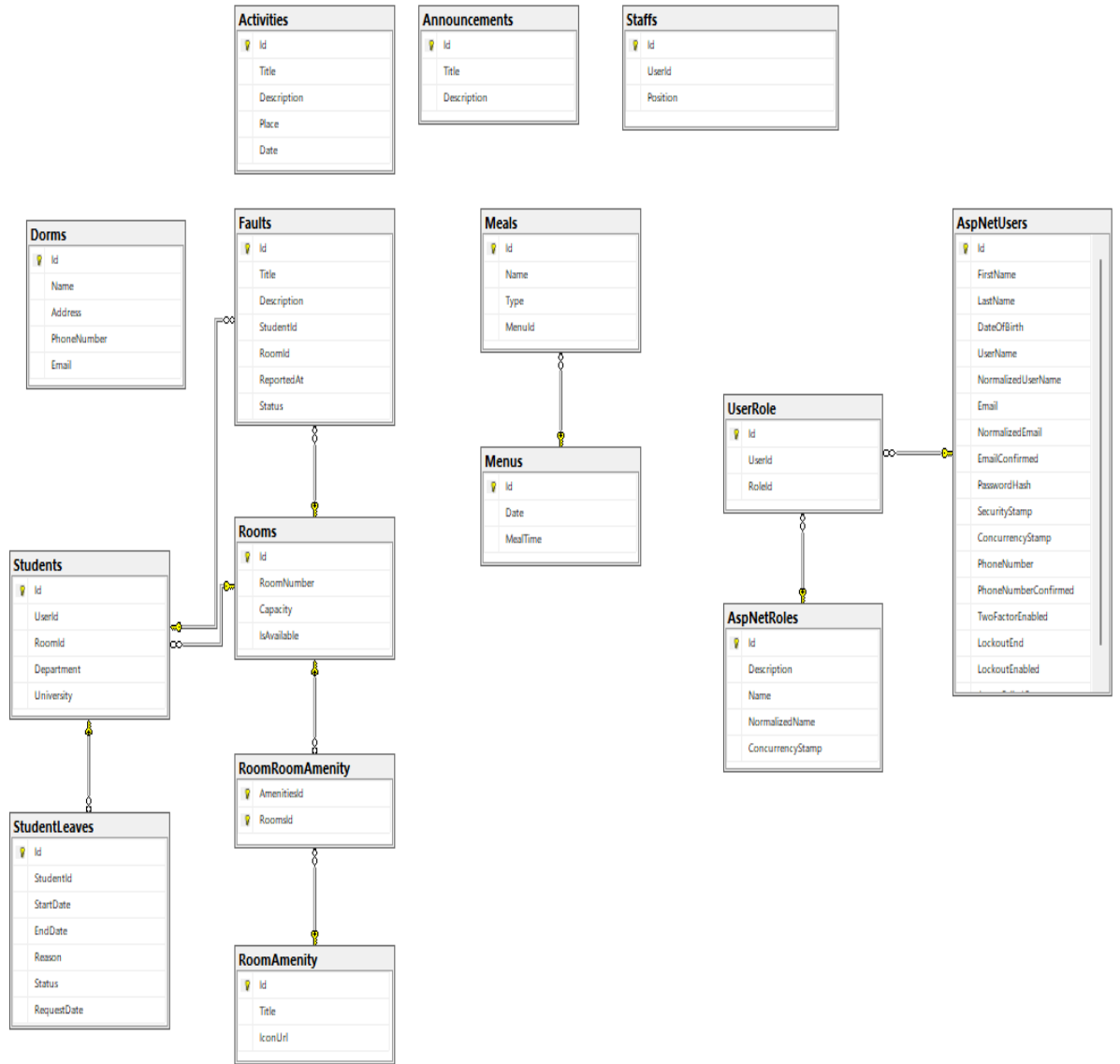
Response body

```
{
  "data": [
    {
      "id": 2,
      "userId": 3,
      "fullName": "Mukaddes Savlı",
      "roomId": 1,
      "department": "Yönetim Bilişim Sistemleri",
      "university": "Aksaray Üniversitesi"
    }
  ],
  "errorMessage": null
}
```

Görsel_3

Veritabanı Şeması:

Örnek şema “Görsel_4” de gösterilmiştir.



Görsel_4

Örnek kod yapılarına aşağıdaki görsellerde yer verilmiştir.

```
11 references
public class GenericRepository<T, TId>(AppDbContext context) : IGenericRepository<T, TId> where T : BaseEntity<TId> where TId : struct
{
    protected AppDbContext Context = context;

    private readonly DbSet<T> _dbSet = context.Set<T>();

    10 references
    public async ValueTask AddAsync(T entity)=>await _dbSet.AddAsync(entity);

    1 reference
    public Task<bool> AnyAsync(TId id)=> _dbSet.AnyAsync(x => x.Id.Equals(id));

    2 references
    public Task<bool> AnyAsync(Expression<Func<T, bool>> predicate)
    {
        return _dbSet.AnyAsync(predicate);
    }
}

10 references
public void Delete(T entity)=>_dbSet.Remove(entity);

9 references
public Task<List<T>> GetAllAsync()=>_dbSet.ToListAsync();

1 reference
public Task<List<T>> GetAllPagedAsync(int pageNumber, int pageSize)
{
    return _dbSet.Skip((pageNumber - 1) * pageSize).Take(pageSize).ToListAsync();
}

31 references
public ValueTask<T?> GetByIdAsync(int id)=>_dbSet.FindAsync(id);

12 references
public void Update(T entity)=>_dbSet.Update(entity);

1 reference
public IQueryable<T> Where(Expression<Func<T, bool>> predicate)=> _dbSet.Where(predicate).AsQueryable().AsNoTracking();
}
```

- Varlıklara ait repository sınıfları “Generic Repository” class’ını kullanmaktadır. Bu sayede kod tekrarından büyük oranda kaçılmıştır.

```

1 reference
public class FaultService(IFaultRepository faultRepository,
    IUnitOfWork unitOfWork,
    IUserService userService
):IFaultService
{
    2 references
    public async Task<ServiceResult<FaultDto>> GetByIdAsync(int id)
    {
        var fault = await faultRepository.GetByIdAsync(id);
        if (fault == null)
            return ServiceResult<FaultDto>.Fail("Not found");

        var fullName = await userService.GetFullNameByIdAsync(fault.Student.UserId);

        //manually mapping
        var faultDto = new FaultDto(
            fault.Id,
            fault.Title,
            fault.Description,
            fault.StudentId,
            fault.RoomId,
            fullName ?? "Unknown"
        );

        return ServiceResult<FaultDto>.Success(faultDto);
    }

    2 references
    public async Task<ServiceResult<CreateFaultResponse>> CreateAsync(CreateFaultRequest request)
    {
        // Create new Fault entity manually
        var fault = new Fault()
        {
            Title = request.Title,
            Description = request.Description,
            StudentId = request.StudentId,
            RoomId = request.RoomId,
            Status = FaultStatus.Pending,
        };

        await faultRepository.AddAsync(fault);
        await unitOfWork.SaveChangesAsync();

        return ServiceResult<CreateFaultResponse>.Success(new CreateFaultResponse(fault.Id));
    }
}

```

- Örnek servis classı yukarıdaki gibidir. “Application katmanında bulunan servis yapıları dto alıp dto dönmektedir. Bu sayede veri bütünlüğü korunmuştur.

```

0 references
public class FaultsController(IFaultService faultService) : BaseController
{
    [HttpGet]
    0 references
    public async Task<IActionResult> GetAll() =>
        CreateActionResult(await faultService.GetAllListAsync());

    [HttpGet("{id:int}")]
    0 references
    public async Task<IActionResult> GetById(int id) =>
        CreateActionResult(await faultService.GetByIdAsync(id));

    [HttpPost]
    0 references
    public async Task<IActionResult> Create(CreateFaultRequest request) =>
        CreateActionResult(await faultService.CreateAsync(request));

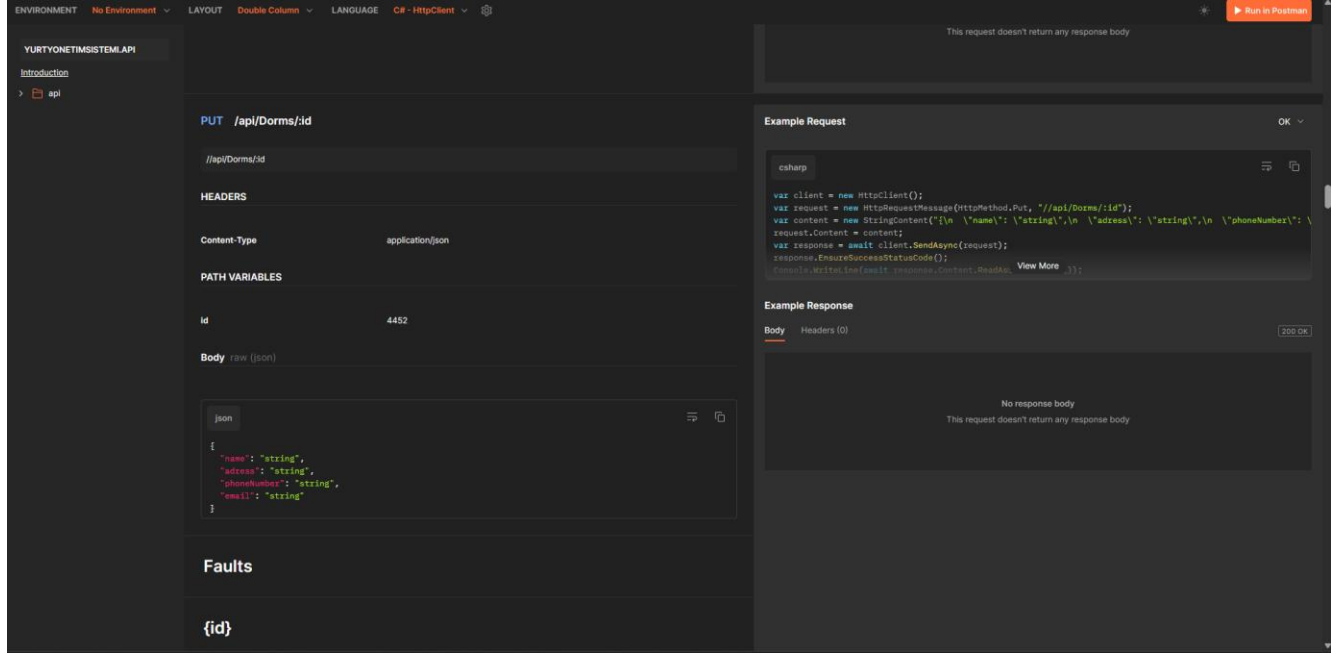
    [HttpPut("{id:int}")]
    0 references
    public async Task<IActionResult> Update(int id, UpdateFaultRequest request) =>
        CreateActionResult(await faultService.UpdateAsync(id, request));

    [HttpDelete("{id:int}")]
    0 references
    public async Task<IActionResult> Delete(int id) =>
        CreateActionResult(await faultService.DeleteAsync(id));
}

```

- Controller yapıları olabildiğince temiz bırakılmıştır. Metod ve endpoint isimlerinde best practice yaklaşımlar tercih edilmiştir.

Kodların tamamı rapor sonunda GitHub repo linki olarak bırakılacaktır.



- <https://documenter.getpostman.com/view/37005138/2sB3dHWYnb> (API Dökümantasyon Linki)
- <https://github.com/suleymannmeral/dormManagementSystem> (GitHub Repo Linki)